TASK 7 – OVERRIDE AND SPECIALISE BEHAVIOUR

1. Introduction

Method overriding is an important feature of object-oriented programming that allows a subclass to provide its own implementation of a method already defined in its superclass. It supports runtime polymorphism and enables specialized behaviour in inherited classes.

In the Campus Smart-Parking System, different vehicle categories have different parking rates. Therefore, method overriding is used to implement vehicle-specific fee calculations while maintaining a common structure through the Vehicle superclass.

2. Methods Overridden in the Vehicle Hierarchy

The `calculateFee()` method is specialized according to the vehicle category.

The following classes are considered:

1. Vehicle – Superclass

- Defines the common structure for all vehicles.
- Stores the registration number.
- Declares `calculateFee(int minutes)` as an abstract method.
- Provides common implementations of `toString()`, `equals()` and `hashCode()`.

2. TwoWheeler – Subclass

- Overrides `calculateFee(int minutes)`.
- Calculates the parking fee using a rate of Rs. 10 per hour.
- Provides vehicle-specific fee calculation.

3. Car – Subclass

- Overrides `calculateFee(int minutes)`.
- Calculates the parking fee using a rate of Rs. 30 per hour.

4. ElectricCar – Subclass

- Inherits the parking fee calculation from Car.
- Uses the car parking rate.
- Can provide additional charging-related functionality separately.

Example of Method Overriding

```
class TwoWheeler extends Vehicle {
    @Override
    public double calculateFee(int minutes) {
        return billableHours(minutes) * 10;
    }
}

class Car extends Vehicle {
    @Override
    public double calculateFee(int minutes) {
        return billableHours(minutes) * 30;
    }
}
```

Both subclasses implement the same method signature but provide different calculations according to their vehicle categories.

This avoids duplicating the entire vehicle structure while allowing specialized behaviour.

3. Use of the @Override Annotation

The `@Override` annotation indicates that a method is intended to override a method declared in its superclass.

Advantages:

- Helps the compiler verify the overriding method.
- Detects incorrect method signatures.
- Improves code readability.
- Makes the inheritance relationship clear.
- Prevents accidental method overloading when overriding is intended.

Rules of Method Overriding

1. The method must have the same name and parameter list as the superclass method.
2. The return type must be the same or covariant.
3. The access level cannot be more restrictive than the overridden method.
4. Private methods cannot be overridden.
5. Final methods cannot be overridden.
6. Static methods are hidden rather than overridden.
7. The `@Override` annotation helps verify the correctness of the implementation.

Therefore, `calculateFee(int minutes)` must retain the same method signature in the subclasses.

4. Appropriate Use of the final Keyword

The `final` keyword restricts modification and is useful for maintaining consistency and protecting important parts of the system.

A. Final Variable

The registration number should be declared final because it uniquely identifies a vehicle and should not change after object creation.

```
private final String regNo;
```

This prevents reassignment of the registration number after initialization.

B. Final Method

A method such as `getRegNo()` can be declared final if its implementation must remain unchanged in all subclasses.

```
public final String getRegNo() {
    return regNo;
}
```

This prevents subclasses from overriding the method.

C. Final Class

A class can be declared final when inheritance must be prevented.

```
final class ParkingReceipt {
}
```

However, the Vehicle class should not be final because it is designed to act as a superclass for TwoWheeler, Car and other vehicle categories.

Thus, final is applied selectively to protect important data and behaviour without restricting necessary inheritance.

5. Overriding Object Class Methods

Java provides the `Object` class as the root superclass of all classes. Important methods such as `toString()`, `equals()` and `hashCode()` can be overridden to provide meaningful vehicle-specific behaviour.

A. Overriding toString()

The default `toString()` method displays the class name followed by a hash-code representation.

For the parking system, it is more useful to display the vehicle type and registration number.

```
@Override
public String toString() {
    return getClass().getSimpleName() +
           "{regNo='" + regNo + "'}";
}
```

Example output:

```
Car{regNo='TN02B2'}
```

This makes vehicle information easier to understand during debugging and monitoring.

B. Overriding equals()

The `equals()` method determines whether two objects should be considered logically equal.

In the parking system, the registration number uniquely identifies a vehicle. Therefore, two Vehicle objects with the same registration number are considered equal, irrespective of their concrete subclass.

```
@Override
```

```
public boolean equals(Object obj) {
    if (this == obj)
        return true;

    if (!(obj instanceof Vehicle))
        return false;

    Vehicle other = (Vehicle) obj;
    return regNo.equals(other.regNo);
}
```

Example:

```
Car c1 = new Car("TN02B2");
ElectricCar c2 = new ElectricCar("TN02B2");

System.out.println(c1.equals(c2));
```

Output:

```
true
```

Both objects have the same registration number, so they are treated as logically equal.

This behaviour is suitable when registration numbers are globally unique across all vehicle categories.

C. Overriding hashCode()

The `hashCode()` method generates an integer hash value for an object. It must be consistent with the `equals()` method.

```
@Override
public int hashCode() {
    return regNo.hashCode();
}
```

If two Vehicle objects are equal according to `equals()`, they must produce the same hash code.

This is important when vehicle objects are stored in hash-based collections such as HashSet and HashMap.

6. Alternative Concept – Using a switch Statement Instead of Method Overriding

An alternative approach is to implement a single `calculateFee()` method in the Vehicle class using a vehicle type field and a switch statement.

Instead of creating separate fee calculation methods in each subclass, the system checks the vehicle category and selects the corresponding parking rate.

Example Implementation

```
double calculateFee(int minutes, String type) {

    int hours = minutes / 60;

    if (minutes % 60 != 0) {
        hours++;
    }

    switch (type) {
        case "TwoWheeler":
            return hours * 10;

        case "Car":
            return hours * 30;

        case "ElectricCar":
            return hours * 30;

        default:
            throw new IllegalArgumentException(
                "Invalid vehicle type");
    }
}
```

The method determines the billable hours and selects the corresponding rate based on the vehicle type.

Comparison Between Method Overriding and switch

Criteria	Method Overriding	switch Statement
Design	Each subclass provides its own implementation	One method handles all vehicle types
Fee calculation	Specialized subclass method	Centralized conditional logic
Extensibility	New subclass can provide its own implementation	Existing switch logic must be modified
Maintainability	Easier to maintain as categories increase	Central method becomes longer
Code organization	Behaviour is distributed among relevant classes	Behaviour is centralized
Polymorphism	Supports runtime polymorphism	Depends on explicit type checking

Time Complexity Analysis

Let k represent the total number of vehicle types.

Method Overriding:

- Runtime method dispatch selects the appropriate implementation.
- Fee calculation requires a constant number of arithmetic operations.
- Time complexity for one fee calculation is O(1).

switch Statement:

- The system checks the vehicle type and selects the corresponding case.
- In a general sequential comparison, the worst-case selection may require up to k checks.
- Therefore, the worst-case conceptual selection complexity is O(k).

- The fee calculation itself takes O(1) time.

Effect of Adding a New Vehicle Type

Method Overriding:

A new vehicle category can be introduced by creating another subclass and implementing its specific fee calculation method. Existing subclasses and the common Vehicle structure generally remain unchanged.

switch Statement:

The existing switch statement must be modified to include the new vehicle category and its pricing rule. The vehicle type definition and related validation may also require changes.

This increases maintenance effort and creates a centralized dependency.

7. Conclusion

Method overriding provides an effective way to implement specialized parking fee calculations for different vehicle categories in the Campus Smart-Parking System.

The `@Override` annotation improves correctness, while the `final` keyword protects important fields and methods. Overriding `toString()`, `equals()` and `hashCode()` provides meaningful object representation and consistent equality behaviour based on vehicle registration numbers.

Compared with a switch-based implementation, method overriding provides better separation of responsibilities, supports runtime polymorphism and makes the system easier to extend when new vehicle categories are introduced.

Therefore, method overriding is preferred for the proposed parking system because it improves code reusability, maintainability and extensibility.